What is Outliers

আউটলায়ার কী?

আউটলায়ার হলো সেই সব ডেটা পয়েন্ট যারা আপনার ডেটাসেটের অধিকাংশ ডেটা থেকে অনেক দূরে অবস্থান করে।

উদাহরণ:

> ধরা যাক, একটি ক্লাসের বেশিরভাগ শিক্ষার্থীর নম্বর ৬০-৭০এর মধ্যে।
> হঠাৎ একজন শিক্ষার্থীর নম্বর ৯৯ → এই ৯৯ হলো আউটলায়ার।
> কারণ, এটি সাধারণ প্যাটার্নের বাইরে।

একইভাবে:

> বয়সের ক্ষেত্রে: ক্লাস ৮-এ অধিকাংশের বয়স ১৩-১৪, কিন্তু কেউ ২০ বছর → আউটলায়ার।
> আইকিউর ক্ষেত্রে: অধিকাংশের আইকিউ ১২০-১৩০, কিন্তু কারও ১৮০ → আউটলায়ার।

বাস্তব উদাহরণ: বেতন ও আউটলায়ার

ধরা যাক, একটা কোম্পানির ৪ জন কর্মচারীর বেতন (হাজার টাকায়):

> ৩০, ৩০, ৩০, ৩০ → গড় বেতন = ৩০
> এখন সিইওর বেতন ২০০ (লাখ টাকায় ২) → নতুন ডেটা: ৩০, ৩০, ৩০, ২০০
> নতুন গড় = (৩০+৩০+৩০+২০০)/৪ = ৭২.৫
> অর্থাৎ, একটি আউটলায়ার গড়কে অনেক বদলে দিল।

একই প্রভাব পড়ে স্ট্যান্ডার্ড ডেভিয়েশন, ভ্যারিয়েন্স, রিগ্রেশন লাইন ইত্যাদিতে।

মেশিন লার্নিং-এ আউটলায়ার কেন সমস্যা?

লিনিয়ার রিগ্রেশনের উদাহরণ:

> ধরা যাক, X = পড়ার ঘন্টা, Y = মার্কস।
> বেশিরভাগ শিক্ষার্থী ৪-১৬ ঘন্টা পড়ে, মার্কস ৪০-৮০ পায়।
> রিগ্রেশন মডেল একটি লাইন ফিট করে যা সব ডেটার কাছাকাছি যায়।

কিন্তু যদি একটি আউটলায়ার থাকে: ২০ ঘন্টা পড়ে, মার্কস ২০০?

→ তাহলে লাইনটি আউটলায়ারের দিকে টেনে যায়।

→ ফলে মডেলটি বায়াসড **(Bias)** হয়ে যায়।

→ নতুন ডেটা দিলে প্রেডিকশন ঠিকমতো করে না।

মূল কথা: আউটলায়ার মডেলকে ভুল শেখায় **(Overfitting** বা **Bias** তৈরি হয়**)**।

আউটলায়ার ডিটেক্ট করার পদ্ধতি

1️⃣ Z-Score পদ্ধতি

> Z-Score বলে দেয়, একটি ডেটা পয়েন্ট গড় থেকে কত স্ট্যান্ডার্ড ডেভিয়েশন দূরে আছে।
> সাধারণত,
> **Z-Score > 3** বা **< -3** হলে সেটি আউটলায়ার (বেল-কার্ভ অনুযায়ী)।
> উপযুক্ত যখন: ডেটা প্রায় নরমাল ডিস্ট্রিবিউশনে থাকে।

সূত্র: Z = (X - Mean) / Standard Deviation

2️⃣ IQR (Interquartile Range) পদ্ধতি

১. ডেটা সাজাও (ছোট থেকে বড়)।

২. Q1 (২৫% পয়েন্ট) ও Q3 (৭৫% পয়েন্ট) বের করো।

৩. IQR = Q3 - Q1

৪. Lower Bound = Q1 - 1.5 * IQR

৫. Upper Bound = Q3 + 1.5 * IQR

৬. যেসব ডেটা বাউন্ডের বাইরে → সেগুলো আউটলায়ার।

উদাহরণ: নম্বরের ডেটার জন্য Q1=৫০, Q3=৮০, IQR=৩০,

তাহলে ৫০-(১.৫×৩০)=৫, ৮০+(১.৫×৩০)=১২৫ → ১২৫-এর বেশি নম্বর আউটলায়ার।

আউটলায়ার নিয়ে করণীয়

✅ পদ্ধতি ১: ডিলিট করা **(Removal / Trimming)**

ব্যবহার করবে যখন আউটলায়ারের সংখ্যা কম।
সরাসরি ডেটাসেট থেকে বাদ দিয়ে দাও।
সতর্কতা: অনেক ডেটা দিলে তথ্য কমে যেতে পারে।

✅ পদ্ধতি ২: ক্যাপিং **(Capping / Winsorization)**

আউটলায়ারকে একটা নির্দিষ্ট মানের মধ্যে নিয়ে আসো।
যেমন: ৯৯-এর বদলে ৮৫ বানিয়ে দাও (আপার বাউন্ডের কাছাকাছি)।
ব্যবহার করবে যখন আউটলায়ার অনেক এবং ডেটা কমাতে চাও না।

Outlier handling via Z Score Theory

Z-Score কী?

Z-Score বলে দেয়, একটি ডেটা পয়েন্ট গড় **(Mean)** থেকে কত স্ট্যান্ডার্ড ডেভিয়েশন **(σ)** দূরে আছে।

সূত্র:

$$Z = \frac{X_i - \mu}{\sigma}$$

Xi = নির্দিষ্ট ডেটা পয়েন্ট
μ = ডেটাসেটের গড়
σ = স্ট্যান্ডার্ড ডেভিয়েশন

নরমাল ডিস্ট্রিবিউশন ও Z-Score

নরমাল ডিস্ট্রিবিউশনে (বেল-কার্ভ):

রেঞ্জ (Mean ± Z)	ডেটার শতকরা পরিমাণ
±1σ	≈ 68.3%
±2σ	≈ 95.4%
±3σ	≈ 99.7%

অর্থাৎ, মাত্র **0.3%** ডেটা থাকে **±3σ**-এর বাইরে।

এই 0.3%-কে আমরা আউটলায়ার হিসেবে ধরি।

Z-Score দিয়ে আউটলায়ার ডিটেক্ট করার নিয়ম

যদি $Z > +3$ বা $Z < -3$ হয় → সেটি আউটলায়ার
যদি $-3 \leq Z \leq +3$ হয় → সাধারণ ডেটা

উদাহরণ:

ধরা যাক, গড় = 70, σ = 10

একজন শিক্ষার্থীর নম্বর = 95

Z = (95-70)/10 = 2.5 → আউটলায়ার নয় (কিন্তু কাছাকাছি)

আরেকজনের নম্বর = 110 → Z = 4 → আউটলায়ার ✅

⚠️ Z-Score কখন ব্যবহার করবেন?

✅ প্রযোগযোগ্য: যখন ডেটা প্রায় নরমালি ডিস্ট্রিবিউটেড

❌ প্রযোজ্য নয়: যখন ডেটা স্কিউড **(Skewed)** — যেমন বেতনের ডেটা (বেশিরভাগ কম, কয়েকজন বেশি)

টাইটানিক ডেটাসেটে Age ছিল প্রায় নরমাল, কিন্তু Fare ছিল স্কিউড — তাই Fare-এর জন্য Z-Score সঠিক নাও হতে পারে।

টাইটানিক ডেটাসেটে Z-Score দিয়ে আউটলায়ার ডিটেক্ট করা

ধাপ ১: নিউমেরিক্যাল কলাম চিহ্নিত করা

টাইটানিক ডেটাসেটে নিউমেরিক্যাল কলাম: Age, Fare
Age: বয়স (প্রায় নরমাল)
Fare: ভাড়া (স্কিউড)

ধাপ ২: Z-Score বের করা

প্রতিটি Age এবং Fare মানের জন্য Z-Score ক্যালকুলেট করি।

ধাপ ৩: আউটলায়ার ফিল্টার করা

```
1# সিম্পল সিউডোকোড


2z_scores = (df['Age'] - df['Age'].mean()) / df['Age'].std()
```

```
3outliers = df[abs(z_scores) > 3]
```

ধাপ ৪: পর্যবেক্ষণ

Ageএ দেখা গেল: কিছু খুব ছোট বাচ্চা (যেমন 1 বছর) এবং কিছু বয়স্ক ব্যক্তি (যেমন 80+ বছর) — তারা ±3σ-এর বাইরে → আউটলায়ার
Fareএ দেখা গেল: সর্বোচ্চ ভাড়া 512 ডলার, গড় 32 ডলার — Z-Score অনেক বড় → আউটলায়ার, তবে এখানে Z-Score সঠিক পদ্ধতি নয়, কারণ ডেটা স্কিউড।

Outlier handling via Z score Code

Z-Score দিয়ে আউটলায়ার ডিটেক্ট করার নিয়ম

আউটলায়ার শুধু ট্রেনিং ডেটা **(X_train)** থেকে ডিটেক্ট ও হ্যান্ডেল করবো। টেস্ট ডেটা **(X_test)** থেকে কখনো নয়।

কেন?

টেস্ট ডেটা হলো রিয়েল ওয়ার্ল্ডের প্রতিনিধি। সেখানে আউটলায়ার থাকলেও মডেলকে সেটার ওপর টেস্ট করতে হবে। ট্রেনিং ডেটা পরিষ্কার করে মডেলকে শেখানো হয়, টেস্ট ডেটা অপরিবর্তিত রাখা হয়।

কোড স্টেপ বাই স্টেপ

```
1# 1. Age কলামের Mean ও Std বের করা
2mean_of_age = X_train['Age'].mean()
3std_of_age = X_train['Age'].std()
4
5# 2. নতুন কলামে Z-Score ক্যালকুলেট করা
6X_train['zscore_Age'] = (X_train['Age'] - mean_of_age) / std_of_age
7
8# 3. আউটলায়ার ফিল্টার করা (|Z| > 3)
9outliers_age = X_train[abs(X_train['zscore_Age']) > 3]
10
11# 4. কয়টা আউটলায়ার ও কারা সেগুলো দেখা
12print(len(outliers_age))
13display(outliers_age)
```

আউটলায়ার হিসেবে বয়স্ক ব্যক্তিরা **(৭০, ৭৪, ৮০ বছর)** ধরা পড়েছে।
শিশু/ইনফ্যান্ট **(০-৪ বছর)** আউটলায়ার ধরা পড়েনি – কারণ গড় (mean) যুবকদের দিকে ছিল।

📌 **Z-Score** নির্ভর করে **Mean** ও **Standard Deviation**-এর উপর।

ডেটা skewed হলে বা একদিকে বেশি ঘনীভূত থাকলে Z-Score সঠিক আউটলায়ার ধরতে পারে না।

Fare কলামের জন্য Z-Score

```
mean_of_fare = X_train['Fare'].mean()
std_of_fare = X_train['Fare'].std()

X_train['zscore_Fare'] = (X_train['Fare'] - mean_of_fare) / std_of_fare

outliers_fare = X_train[abs(X_train['zscore_Fare']) > 3]

print(len(outliers_fare))    # আউটপুট: 17
display(outliers_fare)       # 247 থেকে 512 ডলার পর্যন্ত ভাড়া
```

Fare-এর ক্ষেত্রে সমস্যা

গড় ভাড়া ~32 ডলার, কিন্তু সর্বোচ্চ 512 ডলার।
Z-Score ব্যবহার করে ১৭টি আউটলায়ার পাওয়া গেছে।
কিন্তু **Fare** ডিস্ট্রিবিউশন অত্যন্ত স্কিউড – তাই এখানে Z-Score সঠিক পদ্ধতি নয়।

Skewed ডেটার জন্য IQR (Interquartile Range) পদ্ধতি ভালো কাজ করে।

Z-Score vs Median

Mean (গড়) আউটলায়ারের প্রতি অত্যন্ত সংবেদনশীল (sensitive)।
Median (মধ্যমা) আউটলায়ার দ্বারা প্রভাবিত হয় না (robust)।

উদাহরণ:

ডেটা: ১০, ১২, ১৪, ১৬, ১৮ → Mean = ১৪, Median = ১৪

একটি আউটলায়ার যোগ করো: ১০০

নতুন Mean = (১০+১২+১৪+১৬+১৮+১০০)/৬ = ~২৮.৩ (বেশি বেড়েছে)

নতুন Median = (১৪+১৬)/২ = ১৫ (প্রায় অপরিবর্তিত)

👉 তাই স্কিউড ডেটার জন্য **Median**-ভিত্তিক **IQR** পদ্ধতি ভালো।

Outlier Handling via IQR Theory

IQR কী?

IQR হলো তৃতীয় কোয়ার্টাইল **(Q3)** ও প্রথম কোয়ার্টাইল **(Q1)**-এর ব্যবধান।

সূত্র: IQR = Q3 - Q1

কোয়ার্টাইল মানে কী?

কোয়ার্টাইল	শতাংশ
Q1 (25th percentile)	25% ডেটা এর চেয়ে ছোট, 75% বড়
Q2 (Median, 50th percentile)	50% ডেটা ছোট, 50% বড়
Q3 (75th percentile)	75% ডেটা ছোট, 25% বড়

বেশিরভাগ ডেটা Q1 থেকে Q3-এর মধ্যে থাকে।

আউটলায়ার ডিটেক্ট করার সূত্র (IQR পদ্ধতি)

আমরা উপরের সীমা (Upper Bound) ও নিচের সীমা (Lower Bound) নির্ধারণ করি:

Lower Bound} = Q1 - 1.5 * IQR Upper Bound = Q3 + 1.5 * IQR

এর নিচে যারা → **Low Outlier**
এর উপরে যারা → **High Outlier**

কেন 1.5?

এটি একটি স্ট্যান্ডার্ড থ্রেশহোল্ড।

1.5 ব্যবহার করলে অধিকাংশ স্বাভাবিক ডেটা বাইরে যায় না।
বড় করলে (যেমন 3) কম আউটলায়ার পাবে।
ছোট করলে (যেমন 1) বেশি আউটলায়ার পাবে।

গ্রাফিক্যাল ব্যাখ্যা (Boxplot বা Whisker প্লট)

```
Lower Bound          Q1     Median     Q3          Upper Bound
   |                 |        |        |                |
   |-----------------|--------|--------|----------------|
   ↓                 ↓        ↓        ↓                ↓
[Outliers]    (আইকিউআর-এর মধ্যে)          [Outliers]
```

বক্সের ভিতর Q1 থেকে Q3
বক্সের বাইরে দুটি হুইস্কার (whisker) – এরা Lower Bound ও Upper Bound পর্যন্ত যায়
হুইস্কারের বাইরে যারা → আউটলায়ার

ধাপে ধাপে ক্যালকুলেশন

মনে করি, একটি ডেটাসেট:

Age = [10, 12, 14, 15, 16, 18, 20, 22, 24, 30, 80]

ধাপ ১: Q1 ও Q3 বের করো

সাজানো ডেটা: 10, 12, 14, 15, 16, 18, 20, 22, 24, 30, 80
Q1 = 14 (3rd element)
Q3 = 24 (9th element)

ধাপ ২: IQR বের করো

IQR = 24 - 14 = 10

ধাপ ৩: Bound বের করো

Lower = 14 - (1.5 * 10) = 14 - 15 = -1 Upper = 24 + (1.5 * 10) = 24 + 15 = 39

ধাপ ৪: আউটলায়ার চিহ্নিত করো

ডেটা: 10, 12, 14, 15, 16, 18, 20, 22, 24, 30, 80
Lower = -1 → কোনো ডেটা এর নিচে না
Upper = 39 → **80 > 39** → 80 আউটলায়ার ✅

এখানে 80 বছর বয়সী ব্যক্তি আউটলায়ার।

Outlier Handling via IQR Code

IQR পদ্ধতিতে আউটলায়ার ডিটেকশন

মূল ধারণা

IQR = Q3 - Q1

Outlier: যেসব ডেটা Q1 – 1.5×IQR-এর নিচে বা Q3 + 1.5×IQR-এর উপরে অবস্থান করে।

Age কলামের জন্য IQR পদ্ধতি

```
# 1. Q1 ও Q3 বের করা
age_Q1 = X_train['Age'].quantile(0.25)
age_Q3 = X_train['Age'].quantile(0.75)

# 2. IQR নির্ণয়
age_IQR = age_Q3 - age_Q1

# 3. আউটলায়ার সীমা নির্ধারণ
age_minimum = age_Q1 - 1.5 * age_IQR
age_maximum = age_Q3 + 1.5 * age_IQR

print(age_minimum, age_maximum)   # আউটপুট: 2.5 , 54.5

# 4. আউটলায়ার ফিল্টার করা
age_outlier_IQR = X_train[(X_train['Age'] < age_minimum) |
(X_train['Age'] > age_maximum)]

print(len(age_outlier_IQR))       # কয়টি আউটলায়ার
```

```
18display(age_outlier_IQR)            # কারা সেগুলো
```

নিম্নসীমা = **2.5** বছর → এর নিচে কোনো বয়স নেই (শিশুরাও 0-2.5 এর মধ্যে থাকলে নরমাল)
উর্ধ্বসীমা = **54.5** বছর → এর উপরের বয়সগুলো (57, 59, 61, 63, 70, 80) আউটলায়ার
Z-Score পদ্ধতিতে 70+ কে আউটলায়ার পেয়েছিলাম, কিন্তু IQR-এ **57**-ও আউটলায়ার ধরা পড়েছে।

⚠️ IQR বেশি সংবেদনশীল (sensitive) – তাই প্রয়োজনে 1.5-এর জায়গায় 3 ব্যবহার করতে পারেন কম আউটলায়ার পেতে।

Fare কলামের জন্য **IQR** পদ্ধতি (স্কিউড ডেটার জন্য উপযুক্ত)

```
1# 1. Q1 ও Q3
2fare_Q1 = X_train['Fare'].quantile(0.25)
3fare_Q3 = X_train['Fare'].quantile(0.75)
4
5# 2. IQR
6fare_IQR = fare_Q3 - fare_Q1
7
8# 3. নিচের সীমা (নেগেটিভ হলে 0 বসানো হয়েছে - ভাড়া ঋণাত্মক হতে পারে না)
9fare_minimum = max(0, fare_Q1 - 1.5 * fare_IQR)
10fare_maximum = fare_Q3 + 1.5 * fare_IQR
11
12print(fare_minimum, fare_maximum)    # আউটপুট: 0 , 64.xxx
13
14# 4. আউটলায়ার ডিটেক্ট
15fare_outlier_IQR = X_train[(X_train['Fare'] < fare_minimum) |
(X_train['Fare'] > fare_maximum)]
16
17print(len(fare_outlier_IQR))         # 96টি আউটলায়ার
18display(fare_outlier_IQR)             # 90, 120, 247, 512 ইত্যাদি
```

উর্ধ্বসীমা ≈ **64** ডলার → এর উপরের সব ভাড়া (90, 120, 247, 512) আউটলায়ার
গড় ভাড়া ছিল **32** ডলার – IQR মিডিয়ান ও কোয়ার্টাইলের উপর ভিত্তি করে কাজ করে, তাই বড় ভাড়াগুলো সঠিকভাবে ধরতে পারে।
Z-Score-এ **17**টি আউটলায়ার পেয়েছিলাম (যেমন 247 ধরে নাই)**IQR**-এ **96**টি পেয়েছে – কারণ স্কিউড ডেটায় IQR বেশি কার্যকর।

Best detector of Outlier and Handling

কোন পদ্ধতি কখন ব্যবহার করবে? (Z-Score vs IQR)

পদ্ধতি	নির্ভর করে	উপযুক্ত ডেটা	কেন?
Z-Score	Mean (গড়) ও Std (বিচ্যুতি)	**নরমাল বা প্রায় নরমাল ডিস্ট্রিবিউশন**	Mean আউটলায়ার-সেনসিটিভ, কিন্তু নরমাল ডিস্ট্রিবিউশনে তা সঠিক সীমা দেয়
IQR	Median (মধ্যমা) ও কোয়ার্টাইল	**স্কিউড (Skewed) ডিস্ট্রিবিউশন**	Median আউটলায়ার-রোবাস্ট, তাই skewed ডেটায় সঠিক সীমা দেয়

ডেটা যদি বেল-শেপ হয় → Z-Score
ডেটা যদি একদিকে লেজ লম্বা হয় (Right/Left Skewed) → IQR

টাইটানিক ডেটাসেটে প্রয়োগ (Age ও Fare)

✅ Age: প্রায় নরমাল ডিস্ট্রিবিউশন

Z-Score ফল: ৫টি আউটলায়ার (৭০–৮০ বছর)
IQR ফল: ৫৪টি আউটলায়ার (৫৭ বছর থেকেই শুরু)
সিদ্ধান্ত: Z-Score ভালো (কম false positive) → ডিলিট করবো

```
1# Z-Score দিয়ে আউটলায়ার ডিলিট (শুধু ট্রেনিং ডেটায়)
```

```
2X_train = X_train[abs(X_train['zscore_Age']) <= 3]
```

এখানে abs(zscore_Age) <= 3 মানে যাদের Z-Score -3 থেকে +3-এর মধ্যে, তারাই থাকবে। বাকি ৫টি চলে যাবে।
ট্রেনিং ডেটা আগে ছিল ৭১২টি → ডিলিটের পর ৭০৭টি।

❌ Fare: Right Skewed ডিস্ট্রিবিউশন

Z-Score ফল: ১৭টি আউটলায়ার (শুধু ১৬৪ ডলারের উপরে কিছু)
IQR ফল: ৯৬টি আউটলায়ার (৬৪ ডলারের উপরে সব ভাড়া)
সিদ্ধান্ত: IQR ভালো (skewed ডেটায় বেশি আউটলায়ার ধরে) → ক্যাপিং **(Capping)** করবো

কেন ডিলিট না করে ক্যাপ?

৯৬টি আউটলায়ার ডিলিট করলে ডেটা অনেক কমে যাবে (নষ্ট হবে)
তাই ওপরের সীমা (Upper Bound)-এর বেশি সব মানকে সীমার মানে বসিয়ে দেওয়া ভালো

```
1# IQR দিয়ে সীমা নির্ধারণ
2fare_minimum = max(0, fare_Q1 - 1.5 * fare_IQR)  # ঋণাত্মক যেন না হয়
3fare_maximum = fare_Q3 + 1.5 * fare_IQR
```

```
4
```

```
5# ক্যাপিং (Clipping)
```

```
6X_train['Fare'] = X_train['Fare'].clip(fare_minimum, fare_maximum)
```

এখন ২৪৭, ২২৭ ইত্যাদি সব ভাড়া fare_maximum (প্রায় ৬৪)-এ চলে যাবে।

X_train['Fare'].max() চেক করলে দেখাবে ৬৪ (আগে ছিল ৫১২)।

Z-Score কেন Fare-এ কম আউটলায়ার দিয়েছে?

Z-Score নির্ভর করে Mean ও Std-এর উপর।
Fare-এ বড় আউটলায়ার (২৪৭, ৫১২) থাকায় **Mean** বেড়ে যায় এবং **Std-**ও বেড়ে যায়।
বড় Std-এর কারণে ডিস্ট্রিবিউশন চ্যাপটা **(flat)** হয়ে যায়, ফলে Z-Score বের করলে অনেক বড় মানও (যেমন ১৬৪) নরমাল সীমার মধ্যে পড়ে যায়।

পরীক্ষা করে দেখুন:

```
1min_zscore_fare = X_train[abs(X_train['zscore_Fare']) <= 3]['Fare'].max()
```

```
2print(min_zscore_fare)    # আউটপুট: ~164
```

অর্থাৎ Z-Score ১৬৪ ডলার পর্যন্ত নরমাল বলছে, অথচ IQR বলছে ৬৪-এর উপরে সব আউটলায়ার।

স্কিউড ডেটায় **IQR-**ই সঠিক।

চূড়ান্ত হ্যান্ডলিং স্টেপ (Age ও Fare-এর জন্য)

কলাম	পদ্ধতি	অপারেশন	কোড স্নিপেট
Age	Z-Score	**ডিলিট**	X_train = X_train[abs(X_train['zscore_Age']) <= 3]
Fare	IQR	**ক্যাপ**	X_train['Fare'] = X_train['Fare'].clip(fare_min, fare_max)

Winsorization Technique Code

উইনসারাইজেশন (Winsorization) / পারসেন্টাইল মেথড কী?

এটি একটি ক্যাপিং টেকনিক যেখানে ডেটার দুই প্রান্তের নির্দিষ্ট পারসেন্টাইলের বাইরের সব মানকে ঐ পারসেন্টাইলের মান দিয়ে প্রতিস্থাপন করা হয়।

উদাহরণ:

1% percentile = 10, 99% percentile = 90

তাহলে 10-এর নিচের সব মান → 10 বানানো হবে

90-এর উপরের সব মান → 90 বানানো হবে

কখন উইনসারাইজেশন ব্যবহার করবেন?

পদ্ধতি	উপযুক্ত ডেটা	এখানে প্রযোজ্য?
Z-Score	নরমাল ডিস্ট্রিবিউশন	❌ না (ডেটা skewed)
IQR	স্কিউড ডিস্ট্রিবিউশন (কিন্তু বক্সপ্লটে আউটলায়ার দেখা যায়)	❌ না (এখানে বক্সপ্লটে আউটলায়ার দেখায়নি)
Percentile	যেকোনো ডিস্ট্রিবিউশন, বিশেষ করে যখন আউটলায়ার স্পষ্ট নয়	✅ হ্যাঁ

মার্কস ডেটাসেটের অবস্থা:

```
sns.kdeplot(data=df, x='maths_marks')  # মাল্টি-পিক, skewed


sns.boxplot(data=df, x='maths_marks')  # কোনো আউটলায়ার দেখায় না


df['maths_marks'].describe()
```

আউটপুট:

mean = 48.95
min = 2
max = 100
25% = 23, 50% = 48, 75% = 74

অস্বাভাবিকতা: এভারেজ 49, কিন্তু কোনো শিক্ষার্থী পেয়েছে ২ (অতি কম) এবং ১০০ (অতি বেশি) – এরা আউটলায়ার। কিন্তু IQR বক্সপ্লট এদের ধরে না, কারণ ডিস্ট্রিবিউশন skewed।

পারসেন্টাইল মেথড স্টেপ বাই স্টেপ

```
# 1. একটি পারসেন্টাইল ঠিক করো (যেমন 12%)
x = 12
x = x / 100   # 0.12

# 2. নিচের সীমা (lower bound) - x-th percentile
min_range = df['maths_marks'].quantile(x)      # 12% percentile

# 3. উপরের সীমা (upper bound) - (1-x)-th percentile
max_range = df['maths_marks'].quantile(1 - x)  # 88% percentile

print(f"min range: {min_range}, max range: {max_range}")
```

কেন (1-x)?

x = 0.12 মানে আমরা নিচের 12% ডেটা বাদ দিতে চাই (ক্যাপ করতে চাই)

তাহলে উপরের দিকেও 12% ডেটা বাদ দিতে হবে – তাই 1 - 0.12 = 0.88 (88% percentile)

```
# 4. ক্যাপিং (Clipping) – মানগুলো নির্দিষ্ট রেঞ্জের মধ্যে আনা
df['maths_marks'] = df['maths_marks'].clip(min_range, max_range)

# 5. ফলাফল দেখা
df['maths_marks'].describe()
```

আউটপুট **(x=12% ধরে):**

min → 9.88 (আগে ছিল 2)
max → 85.36 (আগে ছিল 100)
mean ইত্যাদি পরিবর্তিত হবে

পারসেন্টাইল কীভাবে সিলেক্ট করবেন **(x-**এর মান ঠিক করা**)**

```
# বিভিন্ন percentile চেষ্টা করা
for p in [1, 2, 3, 5, 7, 9, 10, 12, 14, 15]:
    val = df['maths_marks'].quantile(p/100)
    print(f"{p}% → {val}")
```

পর্যবেক্ষণ**:**

1% থেকে 12% পর্যন্ত min_range অনেকটাই একই (প্রায় 2)
12%-এর পরে মান বাড়তে থাকে (মানে কম মার্কের ডেটা ফিল্টার হতে থাকে)
12%-এ min_range = 9.88, max_range = 85.36

ট্রেড-অফ**:**

x ছোট করলে কম ডেটা পরিবর্তন হয় (কম aggressive)
x বড় করলে বেশি ডেটা পরিবর্তন হয় (more aggressive)
বেছে নিতে হবে এমন x যেন অস্বাভাবিক চরম মানগুলো ক্যাপ হয়**,** স্বাভাবিক ডেটা অপরিবর্তিত থাকে

Winsorization Technique Theory

Winsorization কী?

Winsorization হলো একটি ক্যাপিং টেকনিক যেখানে:

ডেটার নিচের দিকের নির্দিষ্ট পারসেন্টাইলের বাইরের সব মানকে ঐ পারসেন্টাইলের মান দিয়ে প্রতিস্থাপন করা হয়
উপরের দিকের নির্দিষ্ট পারসেন্টাইলের বাইরের সব মানকে ঐ পারসেন্টাইলের মান দিয়ে প্রতিস্থাপন করা হয়

লক্ষ্য: আউটলায়ারদের সম্পূর্ণ মুছে ফেলা নয়**,** বরং তাদের **"**টেনে আনা**"** – যাতে ডেটার এসেন্স থাকে, কিন্তু এক্সট্রিম ভ্যালুর প্রভাব কমে।

উদাহরণ দিয়ে বুঝুন (মার্কস ডেটা)

মনে করুন, একটি ক্লাসের মার্কস:

```
115, 32, 45, 67, 81, 92, 100
```

এভারেজ ~ 61
15 খুব কম, 100 খুব বেশি – এরা আউটলায়ার

Winsorization করার পর (1% নিয়ে):

Lower 1st percentile বের করলাম → ধরলাম 20
Upper 99th percentile বের করলাম → ধরলাম 92

তখন:

15 → হয়ে যাবে **20** (ক্যাপ করা হলো নিচের দিকে)
100 → হয়ে যাবে **92** (ক্যাপ করা হলো উপরের দিকে)

✅ ডেটা এখন: 20, 32, 45, 67, 81, 92, 92

চরম মানগুলো চলে গেছে, কিন্তু ডেটার আকার ও সংখ্যা অপরিবর্তিত।

Winsorization পদ্ধতির ধাপসমূহ

একটি পারসেন্টাইল মান **(x)** নির্বাচন করুন (যেমন 1, 2, 5)
Lower bound = x-th percentile
Upper bound = (100-x)-th percentile
Lower bound-এর নিচের সব মানকে → lower bound-এ বসান
Upper bound-এর উপরের সব মানকে → upper bound-এ বসান

```
ধরি x = 1
তাহলে:
lower = 1st percentile
upper = 99th percentile

ডেটা ক্যাপিং:
if value < lower: value = lower
if value > upper: value = upper
```

x-এর মান কীভাবে সিলেক্ট করবেন? (ট্রায়াল-এন্ড-এরার)

নিয়ম:

x যত ছোট হবে, তত কম ডেটা পরিবর্তন হবে (আউটলায়ার কম ক্যাপ হবে)
x যত বড় হবে, তত বেশি ডেটা পরিবর্তন হবে (আউটলায়ার বেশি ক্যাপ হবে)

কৌশল:

ছোট x (যেমন 1) দিয়ে শুরু করুন

মডেলের পারফরম্যান্স দেখুন
যদি এক্সট্রিম আউটলায়ার থেকেই যায়, তাহলে x বাড়ান (2, 3, 5...)
যতক্ষণ পর্যন্ত এক্সট্রিম ভ্যালুগুলো ক্যাপ হচ্ছে, ততক্ষণ বাড়ান
যখন দেখবেন আর বাড়ানোর দরকার নেই (এক্সট্রিম ভ্যালু চলে গেছে), সেখানেই থামুন

⚠️ খুব বেশি x নেবেন না, তাহলে স্বাভাবিক ডেটাও পরিবর্তন হয়ে যাবে।

✅ কেন **Winsorization** ব্যবহার করবেন? (সুবিধা)

সুবিধা	ব্যাখ্যা
ডেটা হারায় না	ডিলিটের মতো পয়েন্ট সরিয়ে ফেলে না, শুধু মান পরিবর্তন করে
ইনটুইটিভ	পারসেন্টাইল ধারণা সবার জানা
যেকোনো ডিস্ট্রিবিউশনে কাজ করে	নরমাল, স্কিউড, মাল্টি-মোডাল – সব জায়গায় চলে
কন্ট্রোলেবল	x পরিবর্তন করে কতটুকু ক্যাপ করবেন সেটা আপনি ঠিক করতে পারেন

⚠️ কোথায় **Winsorization** ব্যবহার করবেন না?

যদি আউটলায়ার স্পষ্ট ভুল ডেটা হয় (যেমন বয়স -5) – তখন ডিলিট করা ভালো
যদি ডেটা খুবই ছোট হয় (যেমন ২০টি পয়েন্ট) – তাহলে ক্যাপিং ডিস্ট্রিবিউশন নষ্ট করতে পারে
Z-Score বা IQR ভালো কাজ করলে সেগুলোই প্রাধান্য পাবে